

Building User Stories and Behavior Driven Development Scenarios with a Strict Set of Concepts: Ontology, Benefits and Primary Validation*

Samedi Heng
UCLouvain
Louvain-la-Neuve, Belgium
samedi.heng@uclouvain.be

Konstantinos Tsilionis
KULeuven
Brussels, Belgium
konstantinos.tsilionis@kuleuven.be

Yves Wautelet
KULeuven
Brussels, Belgium
yves.wautelet@kuleuven.be

ABSTRACT

Behavior Driven Development (BDD) offers a way to express scenarios, written in structured natural language, on how the system should act to fulfill a requirement. Such a test scenario is written together with the requirement; this way these two can be conceived in unison, nested into each other. Lots of templates have been written to construct BDD scenarios and various practices were born out of usage. We fail to find documentation on templates. A strict set of templates with a clear definition of the used keywords aligned with the intends of BDD has been proposed recently in the form of an ontology. The present paper (i) evaluates the ontology on existing BDD scenarios found in the GitHub repository (exogenous validation) and (ii) merges an ontology for user story elements' representation with one for expressing BDD scenarios to evaluate its ability to guide the writing of BDD scenarios (endogenous validation). By linking both through strictly-identified concepts, we (i) provide guidance to the practitioner in the agile requirements engineering phase and (ii) with the adequate tagging of elements during the requirements engineering process we get meta-data allowing to suggest treatments to the BDD scenario (e.g. test automation or forward engineering into a software architecture or even code).

CCS CONCEPTS

• Software and its engineering → Requirements analysis;

KEYWORDS

Behavior Driven Development, User Stories, Test Scenarios, Agile Development, Scrum

ACM Reference Format:

Samedi Heng, Konstantinos Tsilionis, and Yves Wautelet. 2023. Building User Stories and Behavior Driven Development Scenarios with a Strict Set of Concepts: Ontology, Benefits and Primary Validation. In *The 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*, March 27-March 31, 2023, Tallinn, Estonia. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3555776.3577696>

*Produces the permission block, and copyright information

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAC '23, March 27-March 31, 2023, Tallinn, Estonia

© 2023 Association for Computing Machinery.

ACM ISBN 978-1-4503-9517-5/23/03...\$15.00

<https://doi.org/10.1145/3555776.3577696>

1 INTRODUCTION

Among the agile-related processes and techniques appearing over the last 20 years, Behavior Driven Development (BDD) and its ability to define user-oriented scenarios for the validation of requirements have received poor attention in the scientific area. Nevertheless, BDD gives a way to *execute the requirement* depicted in a user story so that the scenario constitutes a valuable extension of the story. Ideally, scenarios are written in conjunction with their corresponding user stories to have an agreement with the user on the requirement and its validation. BDD scenarios are thus supposed to be centered on the user with no supporting technical details [19].

Tsilionis et al. [21, 22] present an ontology depicting the keywords most usually found in BDD templates. By tagging (i.e. associating a defined concept to part of a user story or BDD instance) at modeling time, we invoke on meta-data giving us guidance on how to write the scenario but also (1) the point of view used to write the scenario (user or system behavior required); and (2) the nature of the elements so the way they can be used in the software design. In the former source, the ontology and the way it was built were justified but no application nor validation was proposed. The present paper is aimed to fill this gap by addressing two distinct research questions. **RQ1: What is the representation ability of the BDD ontology on large real life scenario sets?** and **RQ2: How can we use the ontology for building BDD scenarios in conjunction with the one on user stories to furnish guidance in the building of the former and furnish potential synergies?** We refer to the first one as an exogenous validation since the sets were built out of the context of the ontology and the second one as an endogenous one since the aim is to build scenarios with the support of the ontology.

To answer the RQ1, we apply the ontology to evaluate the completeness and accuracy of the templates that can be built out of it through the application of a test set of 10 different projects found in GitHub. To answer the RQ2, we merge the BDD ontology with the one developed in Wautelet et al. [25] allowing to build a unified ontology that can serve as guidance to build user stories and BDD scenarios and see a formal link between the nature of the elements used in both of them. Three experts in agile methods and software testing have also been consulted for the evaluation of the strengths and weaknesses of the unified ontology.

This paper is structured as follows: Section 2 describes the background; it introduces the ontology for user stories and BDD scenarios. Section 3 depicts the validation of the BDD ontology on BDD scenario sets found in GitHub. Section 4 depicts the consolidation of the ontology for BDD scenario templates with the ontology for

User Story templates. Section 5 depicts the endogenous validation made using expert opinions. Section 6 discusses the framework's impact and perspectives. Section 7 discusses the related work and, finally, Section 8 concludes the paper.

2 BACKGROUND

The ontologies presented in Sections 2.1 and 2.2 have been built using a comparable approach. Indeed, by collecting the mostly used keywords in practice to build user stories and BDD scenarios, it has been possible to further conceptualize what each of the dimensions are made of and propose definitions for the used keywords. This way a set of non-redundant and non-overlapping concepts could be distinguished and formalized. This section summarizes the results for both ontologies.

2.1 Ontology for User Stories

Wautelet et al. [25] have built an ontology for designing US templates. Figure 1 represents the ontology allowing to build US templates using the UML class diagram notation. A US template can be designed taking an element from the WHO, WHAT and possibly WHY dimensions. The link between the classes conceptually represents the link from one dimension to the other. Concretely, the unidirectional association from the *Role* to one of the *Capability*, *Task* or *Goal* classes implies that the target class instantiates an element of the WHAT dimension (always tagged as *wants/wants to/needs/can/would like* in the model). Then, the unidirectional association from one of these classes instantiating the WHAT dimension to one of the classes instantiating the WHY dimension (always tagged as *so that* into the model) implies that the target class eventually (since 0 is the minimal cardinality) instantiates an element of the WHY dimension. A US template supported by our model is for instance: *As a <Role>, I would like <Task> so that <Hard-goal>*.

Each concept is associated with a particular syntax (identical to the name of the class in Figure 1) and a semantic. The syntax and semantics of the model are summarized here. As a result of the research conducted in [25], the couples syntax/semantic are the following:

- A **role** is an abstract characterization of the behavior of a social actor within some specialized context or domain of endeavor;
- A **task** specifies a particular way of attaining a goal;
- A **capability** represents the ability of an actor to define, choose, and execute a plan for the fulfillment of a goal, given certain world conditions and in the presence of a specific event;
- A **hard-goal** is a condition or state of affairs in the world that the stakeholders would like to achieve;
- A **soft-goal** is a condition or state of affairs in the world that the actor would like to achieve. But unlike a hard-goal, there are no clear-cut criteria for whether the condition is achieved, and it is up to the developer to judge whether a particular state of affairs in fact achieves sufficiently the stated soft-goal.

The goal should necessarily be a hard or a soft-goal, that is why it is represented as an interface in the UML class diagram.

2.2 Ontology for BDD Test Scenarios

Tsilionis et al. [21] have built an ontology for defining BDD scenarios; it is represented in Figure 2 using the UML class diagram

notation. As can be seen in the Figure, two kind of concepts can be distinguished: *user-driven* and *system-driven* scenarios. The former refer to human-related concepts, the later concepts refer to software related concepts.

Each concept is associated with a particular syntax (identical to the name of the class in Figure 1) and a semantic. The syntax and semantics of the model are summarized here. As a result of the research conducted in [25], the couples syntax/semantic are the following:

- **Context:** “The system context is the part of the system environment that is relevant for the definition as well as the understanding of the requirement of a system to be developed” [17];
- **Precondition:** “A required precondition captures a permission to perform the operation when the condition is true” [11];
- **State:** “A set of preconditions or postconditions” (custom definition);
- **User_Behavior:** “The steps an actor can take to achieve his goal or a system’s response” [5];
- **System_Behavior:** “Observable activity of a system, measurable in terms of quantifiable effects on the environment whether arising from internal or external stimulus” [18];
- **Outcome:** “The business benefits that will result from meeting the business needs and the end state desired by stakeholders” [9];
- **Postcondition:** “A required postcondition captures an additional condition that must hold after any application of the operation” [11].

In order to treat the ontology elements in a generic manner, we build 3 super classes: respectively one for the *GIVEN*, *WHEN* and *THEN* dimensions.

3 RQ1: ABILITY OF THE BDD ONTOLOGY TO CHARACTERIZE SCENARIO SETS

The ontology for building BDD scenarios, as presented in Figure 2 is evaluated here on the basis of its **coverage** (i.e., *is each element of the ontology required?*) and **completeness** (i.e., *are there any more elements required?*). The former evaluates whether all of the ontology’s elements are required while the latter examines whether there are any missing elements (i.e., instances that cannot be assigned to an element present in the ontology because no matching semantic as defined in Section 2.2 could be fitted). Secondly, we checked whether the *hypothesis* stating that BDD scenarios can be classified in *user-driven* and *system-driven* types, stands true.

To perform the validation, 10 projects describing BDD test scenarios were selected randomly from the GitHub’s repository; these accounted for **356 test scenarios** in total. The details of the evaluating process and the actual dataset are presented in the Appendix¹ due to the space limitations.

To perform the validation, we searched the GitHub’s repository for projects where BDD test scenarios have been used. We used keywords ‘BDD’ and ‘BDD Test’ to search in the GitHub repository. List of projects is provided as result. We went through project by project to evaluate if test scenarios are effectively used; then the

¹The dataset as well as all the other appendices for the evaluation of the ontology can be found at: <https://data.mendeley.com/datasets/svmcxt5z5f/1>

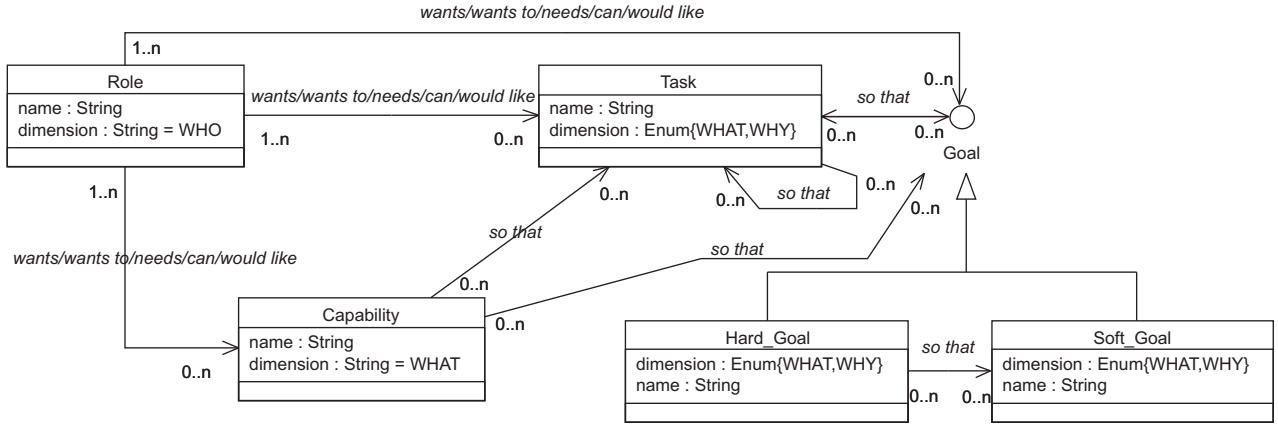


Figure 1: Unified Model for User Story Descriptive Concepts (from [25]).

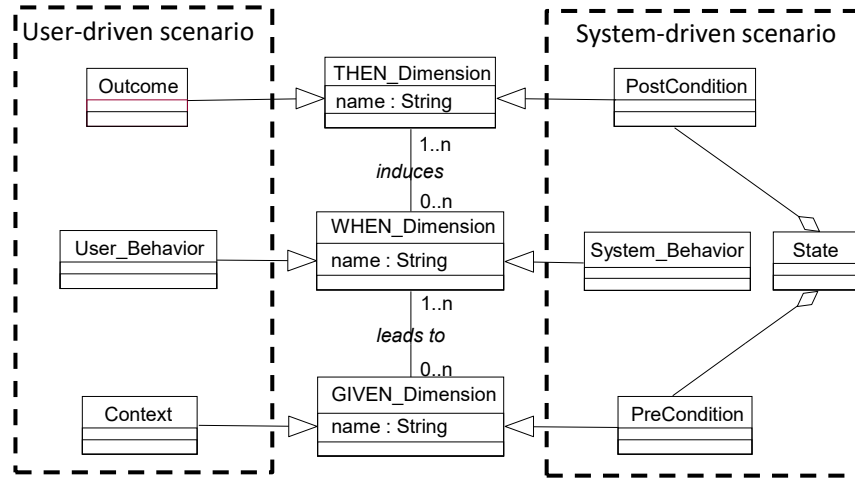


Figure 2: Ontology for BDD Test Scenarios (from [21]).

project could be selected. We selected only the first 10 projects found from the repository. The evaluation was systematically conducted by one of the authors and double-checked by another one. After a careful examination of each BDD scenario, each dimension's instance has been related to the best matching semantic (to tag it as an element of the ontology). Table 1 provides the result of the evaluation. It accounts for 355 test scenarios in total.

Our analysis shows that, considering the test scenarios provided by these 10 projects, we have full coverage; this means that, in regards to these projects, no ontology element is in excess. Additionally, we have found that not every dimension corresponding to the *GIVEN*, *WHEN* and *THEN* canonical form is systematically present in each of these test scenarios. Nevertheless, each time a scenario contains all these dimensions, their instances can always be assigned to one of the defined concepts within the ontology corresponding to a matching semantic as derived from our approach. Also, our aforementioned hypothesis is confirmed; our analysis suggests that every test scenario can always be classified as either a *user-driven* or a *system-driven* one. To be specific, within our test set,

250 test scenarios are user-driven scenarios while the remaining 106 are system-driven.

The test scenarios, as found within these 10 projects, suggest that once the *WHEN* dimension is absent, the BDD scenario is likely to be a *system-driven* one (because it depicts system outputs that we relate as post-conditions). Conversely, once the *WHEN* dimension is present, it constitutes the decisive element making a true segregation between a *user-driven* and a *system-driven* scenario. In other words, it was often hard to discriminate a *Context* from a *Precondition* and an *Outcome* from a *Postcondition*. However, it was easy to distinguish between a *User_Behavior* and a *System_Behavior* in the *WHEN* dimension. For this reason, we point to the discriminating elements as being the *User_Behavior* and the *System_Behavior*; the *Context* is a *Precondition* in a *user-driven* scenario and the *Precondition* is a *Context* in a *system-driven* scenario. Similarly, the *Outcome* is a *Postcondition* in a *user-driven* scenario and the *Postcondition* is an *Outcome* in a *system-driven* scenario.

We also noticed that an instance of *User_Behavior* is often written in the form of *<Role>* followed by an action verb. The *<Role>* is

Table 1: Result of the evaluation of the hypothesis of the ontology.

	Context	User_Behavior	Outcome	PreCon.	System_Behavior	PostCon.	User_driven	System_driven	Number of tested Scenario
Project 1	0	72	0	141	0	136	72	69	141
Project 2	9	5	9	13	13	14	5	18	23
Project 3	0	52	51	67	15	16	52	15	67
Project 4	12	10	12	0	2	0	10	2	12
Project 5	0	24	0	24	0	24	24	0	24
Project 6	19	20	22	0	2	0	20	2	22
Project 7	8	8	8	0	0	0	8	0	8
Project 8	7	7	7	0	0	0	7	0	7
Project 9	0	16	0	16	0	16	16	0	16
Project 10	35	35	35	0	0	0	35	0	35

expressed by using a pronoun, e.g. ‘I’, or a noun phrase, e.g. ‘the user’. Instead, an instance of *System_Behavior* is expressed in the form of a subject which refers to a component or an object of the system, e.g., *the default password*, *the login page*, etc. In general, it is written in a passive form. However, this finding requires a more in depth investigation. We have to build a larger dataset and employ Natural Language Processing (NLP) techniques which is out of the scope of this paper.

In addition, we found that the percentage of concepts’ occurrences of PreCondition (73.88%) is higher than the one for Context (25.00%) for the GIVEN dimension. It is the same for WHEN dimension (70.22% for User_Behaviour and 9.55% for System_Behaviour) and THEN dimension (58.15% for PostCondition and 40.45% for Outcome).

The result shows that all the concepts of the ontology are used (so the completeness element is validated). We found that not every dimension of the canonical form GIVEN, WHEN, THEN is systematically present in the test scenario. When it is, its instance can nevertheless always be assigned to one of the concepts of the ontology (so matching semantic was found). In addition, every scenario can be assigned to one and only one type of scenario: i.e., either *user-driven scenario* or *system-driven scenario*. Therefore, we can conclude that our hypothesis is valid.

Finally, we empirically observed that BDD scenarios are used with user stories in half of the selected projects of the dataset. A user story is tested at least by one scenario; as can be seen in Figure 3, the *WHAT* and *WHY* dimensions in the user story are linked, respectively, to the dimensions *WHEN* and *THEN* of the BDD test scenario. However, the dimension *GIVEN* has no link with any of the dimensions of a user story. We also witnessed occasionally that the BDD’s *THEN* dimension had a link with the user story’s *WHAT* one. This motivated us to consolidate concepts used in test scenarios with the concepts used in user stories as presented in the next section.

4 UNIFYING USER STORY AND BDD SCENARIO ELEMENTS: CONSOLIDATED MODEL

A scenario’s purpose is to specify the desired system behavior upon the implementation of the expressed desideratum as described in the *WHAT* dimension of a user story. Therefore, a link can be made between the BDD ontology and the user story one presented in the study of Wautelet et al. [25]. Both were built following a similar methodological approach. This section thus presents the merging of both ontologies.

To simplify the consolidation of the ontologies, we only took the elements of the *WHAT* and *WHY* dimensions of the user story ontology and aggregated them through superclasses (so one for the *WHAT* and the other for the *WHY* dimension). The *WHEN* dimension of the BDD scenarios has been linked with the *WHAT* dimension of the user story elements to show that a *Capability*, a *Task* and a *Goal* can be operationalized through *User_Behavior* or *System_Behavior*. Similarly the *THEN* dimension of the BDD scenarios has been linked to the *WHAT* and *WHY* dimensions of the user story elements to show that an *Outcome* or a *Postcondition* fulfills the elements instances of those classes.

The cohesion between the user story and BDD artifacts is maximum. Both furnish a non-overlapping and complementary documentation to the software development team. User stories are driving agile software development by being the scope elements for sprint content planning and BDD scenarios furnish enough documentation to execute and validate the requirement depicted in the user story. Moreover the user story is not only the higher-level (or aggregated description) of the BDD scenario but it gives also the general context of the BDD scenario. The user story is thus also necessary for the development team to understand under what scope the execution takes place and documents relevant complementary system behavior.

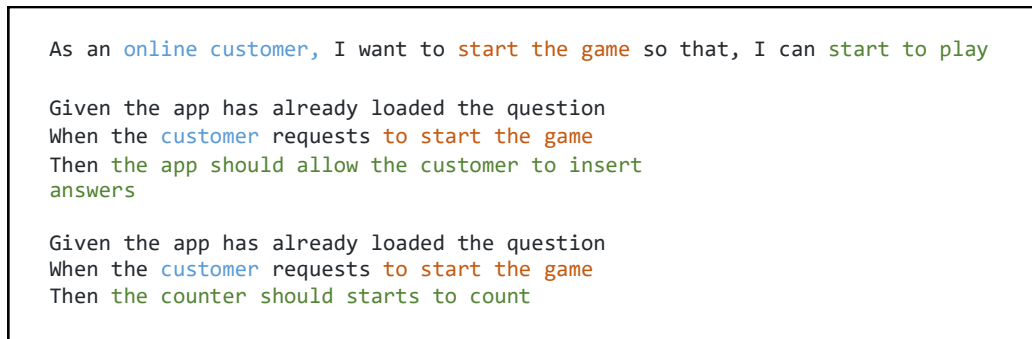


Figure 3: Example of BDD test scenario and User Story.

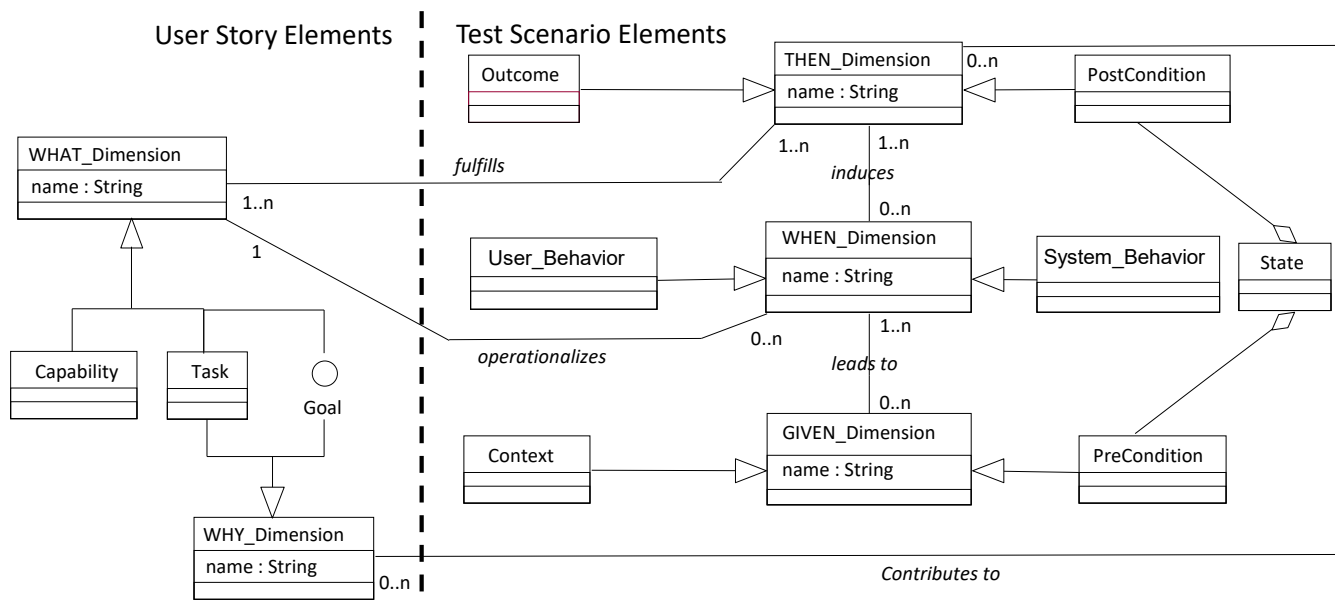


Figure 4: User Stories and Test Scenario Elements: Consolidated Ontology.

5 RQ2: EXPERT OPINIONS ON THE CONSOLIDATED ONTOLOGY AND ITS USE

The unified ontology (Figure 4) has been audited by 3 software development experts in terms of its structure/corresponding parts, understandability, applicability, and ease of use. The first expert holds in PhD in computer science and currently fulfills the role of a researcher/developer. The second expert occupies the role of a team coordinator in a software development team given his extensive experience (+15 years) in the field. The third expert holds a PhD in computer science and is currently occupying the role of an analyst-developer. All three experts were acclimatized to agile methods and test-driven software development processes (mainly via the use of test-driven development and secondly via the use of domain-driven development and BDD); they were encouraged

to consider their business roles and the fit-for-use of the ontology to their daily organizational activities/tasks in order to offer comments, recommendations, concerns, or vulnerabilities about its characteristics.

Each expert participated in an individual interview-session lasting about 90 minutes. The interview protocol that was followed for the evaluation of the unified BDD test scenario ontology was the same for the three experts. In general, the first part was devoted to collecting some background information about the experts' knowledge/expertise and the nature of their interaction with BDD. For the second part, a specific software application was presented to each expert along with a set of 10 user stories describing the user/system requirements for this application. Next, each expert was asked to compose 5 BDD test scenarios corresponding to the subset of the first 5 user stories presented before. Following, a member of the research team proceeded in a full-scale description/explanation of

the ontology of Figure 4 along with its corresponding concepts (i.e., keywords and their semantic allocation). Each expert was asked then to compose 5 BDD scenarios for the remaining 5 user stories of the application while using the ontology as a guiding aid. For the last part of the protocol, each expert could perform a thorough audit of the ontology when it comes to its structure/corresponding parts, understandability, applicability, and ease of use. The experts were encouraged to consider their business roles and the fit-for-use of the ontology to their daily organizational activities/tasks in order to offer comments, recommendations, concerns, or vulnerabilities about the latter's characteristics.

All the experts found the ontology easy to understand and to be providing a benefit in offering a distinction between human-related concepts and system-related concepts within different BDD test scenarios; they all converged that the job of a tester can become easier with the introduction of some clear semantic definitions within the notions used in BDD scenario templates. In the **first expert's** own words *"...after the presentation of the ontology, I have a better grasp of the distinction between the user requirements, and what the state of the system/software before/after a specific event needs to be, for the latter to be in a specific condition validating the user story"*. The expert reported that the above-mentioned distinction made him realize that his job/role falls under the determination of system-driven behaviors and the set-up of quality criteria to better test the functionalities of these systems. Finally, the expert acknowledged that the ontology seems a bit generic and lacks some pivotal details that would make it more customized to the needs of his team. So he values it as a generic guidance that could be customized for a specific team. He also admitted that such a customization would require a lot of time investment on behalf of his team and they would probably be unwilling to put in that extra effort in order to bring a new element/technique in their way of working; the expert admitted that this very last remark is more related to people's natural resistance to change management than an actual drawback of the ontology itself.

The **second expert** expressed the opinion that the ontology can be targeted towards novice testers where a preliminary conceptual differentiation between user-, and system-driven scenarios would be particularly beneficial for customers and developers given that *"when an approach (a user-driven or a system-driven) is selected, the ontology can give some guidance on how to correspond the script of each scenario to its different dimensions and not to mix both approaches. I have personally seen how easy it is for customers and developers to mix both approaches"*. So, overall, the second expert notes that the aforementioned distinction can be particularly beneficial in terms of getting clients, developers, and testers on the same page in terms of declaring the outcome of a testing process successful (or unsuccessful) when targeted at the validation of a user/system requirement.

The **third expert** found very interesting the use of different keywords that can be corresponded to user-, and system-driven scenarios according to the nature (technical or not) of the user story. The expert found the semantic definitions that accompany these keywords supportive of this primary scenario distinction. In his own words *"the ontology, and hence the two types of scenarios, can be used by different types of test teams or testing purposes. System-driven scenarios can be used by the development team, and user-driven*

scenarios can be used by the unit acceptance testing team or the quality assurance team". In this regard, we notice a discrepancy between the second and third expert; the former sees the ontology more fitting to novice testers while the latter considers the ontology relevant to more experienced testers/developers since they may be more capable of realizing the nature of the scenario they want to produce; in such manner, the ontology's accentuation of the different keywords attributed to user-, and system-driven scenarios would be of help. In this regard, his opinion coincides to the one of the first expert when recognizing that the ontology can be used in a top-down fashion *"as a reference ontology that we can use for writing different tests for different teams. However, since each User Acceptance Testing (UAT) team has a different perspective of testing, different parts of the ontology can be used by different development teams according to the different testing prerequisites that they have"* thus allowing for a bottom-up use of the ontology.

Finally, all experts converged on the fact that the ontology benefits from a visible link between the *WHAT*-, *WHY*- user stories' dimensions and the *GIVEN*-, *WHEN*-, *THEN*- BDD scenarios' ones and that pairing elements found within the user stories with elements found in the BDD scenarios furnish extra context and information on the way things can be expressed in a coarse- and fine-grained manner. The consolidation of the user story and the BDD ontologies is justified.

6 FRAMEWORK IMPACT AND DISCUSSION

Even if the unified BDD ontology has been evaluated on 10 projects and we collected the opinion of 3 experts, a more extensive validation of it and the consolidated version (the one including the user story concepts) still needs to be achieved. This will be done on the basis of a larger test set combined with interviews from practitioners to study the rationale of their BDD scenario expressions and automated NLP techniques. We want to study particularly the pair user story/BDD scenario and develop guidelines to build them in a unified way and, with the use of a specifically developed Computer Aided Software Engineering (CASE) tool, we can trace the link between them and the validation of traditional user stories but also Epic user stories on the basis of multiple BDD scenarios. All in all, this will allow a global linkage between the requirements in terms of user goals and task and their operational realization. Also the use of the meta-data to automate the transformation process of the BDD scenario into code will be further studied.

The overall quality of BDD scenarios will also be further studied with the ontology as main guidance. The ultimate goal is to build a set of rules that can be used to evaluate and guide the writing of BDD scenarios as it has been done for user stories in [13]. Also, we will study how we can build a graphical approach to link user stories decomposition (functional but also non-functional elements) with their realizing scenarios on the basis of the rationale tree approach [26].

Finally, we aim to study the forward engineering (transformation) process of BDD scenarios into an Agent-oriented software design in the same fashion as it has been done for user stories in [24]. A preliminary study to bridge the user story/BDD concepts to behavior-oriented languages like agent-oriented ones is performed in Heng et al. [6]. The pair of information based on the ontology

has been used to generate a software architecture for an agile declination of the MERODE process in Snoeck & Wautelet [20]. The transformation from user stories to UML use cases on the basis of the unified user story ontology was studied in [23], we will further study the transformation of associated BDD scenarios to workflows for automating a possible visual representation.

7 RELATED WORK

As seen, Wautelet et al. [25] addresses the topic of quality improvements in agile requirements by materializing an objective standard in the use of user stories. We implicitly expand that work here in the field of BDD. This ontology becomes the basis for a visual representation of a user story structuring method called the Rational Tree in [26]. The latter uses parts of the i* framework and visual notation to group user stories around common Epics and Themes for recognizing and reducing even further any occurring modeling redundancies during the stages of requirements analysis and design.

To continue, we examined the work of Bjarnason et al. [3]; the authors perform a case study using data from 6 different software developing companies in order to depict the causes/aftermath of a weak traceability between the stage of analysis of newly-introduced requirements and their testing. Before the actual case study, the author present the evaluation of many methods, automated or not, targeted at reducing the disaffiliation between these two stages. The lack of guidelines the number of traces between requirements and test are reduced by linking test cases to user scenarios abstracted from the formal requirements, thus tracing at a higher abstraction level.

Hotomski et al. [7, 8] specify an approach that uses a three-step intervention system to synchronize between changes in the requirements and changes in their corresponding acceptance tests, during the evolution of a software system. Their approach identifies firstly all relevant change patterns in requirements; second, it generates suggestions in natural language about the processing of these changes; third, it disseminates information about the changes and suggestions to the relevant parties. Overall, this particular approach uses a semantic evaluation of words/sentences to estimate a dissimilarity index which corresponds to the discrepancy between old and newly-introduced requirements. When this index transcends a certain threshold, the discrepancy is considered substantial which means that the acceptance tests nested within the old requirement will need to be modified also. Our study uses semantic evaluation but towards a different direction; it is targeted at creating a broader understanding of the used keywords that would guide requirements engineers in building scenarios that fully correspond to the intends of BDD.

Improvements in BDD test scenarios are tackled in Oliveira et al. [14] via a literature review on quality characteristics in agile requirements resulting in a list of prospective attributes that test scenarios should share. This primary list is further scrutinized via a student experiment and later evaluated by BDD practitioners in Oliveira et al. [15]. Based on their evaluation, a test scenario is graded as qualitative when it is *concise*, *testable*, *understandable*, *unambiguous* and *valuable*. Comparably, in Binamungu [2], BDD practitioners are being surveyed on the minimum quality criteria

that BDD test suites should fulfill. The study proposes ultimately four principles supporting the assessment of quality in test suites (i.e., *smaller scenario steps*, *use of the same domain terms*, *use of generic rather technical terms* and *use of a consistent level of abstraction everywhere in the test suite*). It is important to mention that these principles do not seem to surface from the practical experience of the practitioners; rather, the practitioners are provided with a well-scrutinized list of principles, as derived from the existing literature, and are asked to validate it. Overall, these last research strands, appraising whether a test scenario script matches a predetermined checklist of attributes/principles, provide useful insights in defining the objectives of test scenarios a priori; in practice, it can be difficult to recognize and fully instantiate these attributes/principles in the course of writing test scenarios.

Lastly, Li et al. [12] use visual diagrams as requirement interceptors for the creation of automated test scenarios in Cucumber [1, 4, 27]. Their approach involves the use of a software tool which receives UML state machine diagrams [16] as input and transforms them into abstract graphs with initial and final nodes. The number/format of these nodes leads, according to predetermined rules, to the evolution of Cucumber features. While the quality of the scenarios is ensured by the internal configuration rules of the software itself, still it remains a plug-and-play technique that does not discuss any objective criteria for the correctness of these acceptance tests nor gives any guiding criteria on how the latter were perceived. Hence, the issue of analyzing the current format of BDD test scenarios in order to map-out inductively a generic and unified model that could instantiate a test scenario according to BDD requirements has not, to the best of the authors' knowledge, been addressed yet.

8 CONCLUSION

BDD is a technique often used in agile methods aimed to deliver system functionalities best suited to the needs of the end-users while facilitating the cooperation between the roles involved in the entire development process. The focal point of BDD is situated on the definition of acceptance criteria written in a simple non-technical language to validate the user desiderata described, usually, in the form of user stories. These acceptance criteria take the form of test scenarios examining the alignment between the expected and actually performed user and/or system behavior upon the implementation of these functionalities. Despite the multitude of BDD test scenario templates, there is a lack of clearly defined guidelines on how to write fit-for-purpose scenarios. This paper aimed at evaluating a unified and well-defined ontology to better instantiate most BDD scenario templates in conjunction with the user stories they are associated with. Such a structure that includes a set of concepts with syntax and semantic, aspires to assist practitioners to engage in a common and more precise use of BDD scenarios increasing their overall understandability within and outside a development team.

From the dataset applied on the BDD ontology we learned that the coverage and completeness elements are optimal. We also found that not every dimension of the canonical form GIVEN, WHEN, THEN is systematically used in practice. In addition, every scenario

can be assigned to one and only one type of scenario: i.e., either *user-driven scenario* or *system-driven scenario*. The key elements learned out of the expert interviews are that (i) the ontology offers a strong guidance for writing user stories and their corresponding BDD scenarios; (ii) having the opportunity to consult a unique version of how the pair user story/BDD scenario should be defined brings unification in a very fragmented domain and (iii) the ontology would be of help for automated testing of system and user-oriented scenarios. With the use of the ontology and adequate quality level user stories and BDD scenarios, scenario instances with the meta-data can be used as input for automated software tests.

Since the increase in quality of user stories and BDD scenarios written with the support of the ontology remains an open issue, future work includes the design of a controlled experiment examining the easy of use, consistency, completeness, accuracy, and adaptability of the unified test scenario ontology. Such a research design would dictate the separation of a prospective sample of novice modelers in several test groups. One of which would be asked to draft test scenarios from a given user stories' set with the help of our ontology, the other group would be asked to perform the same task but without the assistance of our ontology. They would, however, be provided with examples concerning the use of test scenarios (or other treatments) in order to make a final comparison in the quality of the furnished BDD scenarios and the understandability of the software problem and solution for all test groups.

As far as the forward engineering/transformation abilities from the ontology combining user stories with the BDD scenarios is concerned, some preliminary work has been done on the generation of code constructs. Finally, using meta-data on the type of elements, fragments can be used to generate design elements but also a graphical representation for analysis purpose. For this, we will, in future work, refine the proposal of [24] to integrate the BDD scenario in a tree depicting user stories' elements links and decompositions. A complete formalization and validation of a graphical representation and its use will be the subject of a future communication. Finally, two CASE tools using and supporting the ontology for writing user stories and BDD scenarios are under development: the DesCartes Architect tool for agent-based development and the Merlin-tool to support the Agile-MERODE process.

REFERENCES

- [1] [n. d.]. Cucumber. <https://cucumber.io/>. Accessed: 2020-10-12.
- [2] Leonard Peter Binamungu, Suzanne M. Embury, and Nikolaos Konstantinou. 2020. Characterising the Quality of Behaviour Driven Development Specifications. In *Agile Processes in Software Engineering and Extreme Programming - 21st International Conference on Agile Software Development, XP 2020, Copenhagen, Denmark, June 8-12, 2020, Proceedings (Lecture Notes in Business Information Processing)*, Viktoria Stray, Rashina Hoda, Maria Paasivaara, and Philippe Kruchten (Eds.), Vol. 383. Springer, 87–102. https://doi.org/10.1007/978-3-030-49392-9_6
- [3] Elizabeth Bjarnason, Per Runeson, Markus Borg, Michael Unterkalmsteiner, Emelie Engström, Björn Regnell, Giedre Sabaliauskaite, Annabella Loconsole, Tony Gorschek, and Robert Feldt. 2014. Challenges and practices in aligning requirements with verification and validation: a case study of six companies. *Empirical software engineering* 19, 6 (2014), 1809–1855.
- [4] David Chelmsky, Dave Astels, Bryan Helmkamp, Dan North, Zach Dennis, and Aslak Hellesoy. 2010. The RSpec book: Behaviour driven development with RSpec. *Cucumber, and Friends, Pragmatic Bookshelf* 3 (2010), 25.
- [5] Mike Cohn. 2004. *User Stories Applied: For agile software dvlpt*. Addison-Wesley Pro.
- [6] Samedi Heng, Monique Snoeck, and Konstantinos Tsilionis. 2022. Building a Software Architecture out of User Stories and BDD Scenarios: Research Agenda. In *Short Paper Proceedings of the First International Workshop on Agile Methods for Information Systems Engineering (Agil-ISE 2022) co-located with the 34th International Conference on Advanced Information Systems Engineering (CAiSE 2022)*, Leuven, Belgium, June 6, 2022 (CEUR Workshop Proceedings), Palash Bera, Fabiano Dalpiaz, and Yves Wautelet (Eds.), Vol. 3134. CEUR-WS.org, 40–46.
- [7] Sofija Hotomski, Eya Ben Charrada, and Martin Glinz. 2018. Keeping Evolving Requirements and Acceptance Tests Aligned with Automatically Generated Guidance, See [10], 247–264. <https://doi.org/10.1007/978-3-319-77243-1>
- [8] Sofija Hotomski and Martin Glinz. 2019. GuideGen: An approach for keeping requirements and acceptance tests aligned via automatically generated guidance. *Info. and SW Tech.* 110 (2019), 17–38.
- [9] Kevin Brennan IIBA. 2009. *A Guide to the Bus. Anal. Body of Knowledge*. Intl. Inst. of Bus. Anal.
- [10] Erik Kamsties, Jennifer Horkoff, and Fabiano Dalpiaz (Eds.). 2018. *Requirements Engineering: Foundation for Software Quality - 24th International Working Conference, REFSQ 2018, Utrecht, The Netherlands, March 19-22, 2018, Proceedings. Lecture Notes in Computer Science*, Vol. 10753. Springer. <https://doi.org/10.1007/978-3-319-77243-1>
- [11] Emmanuel Letier and Axel Van Lamsweerde. 2002. Deriving operational software specifications from system goals. *ACM SIGSOFT Software Engineering Notes* 27, 6 (2002), 119–128.
- [12] Nan Li, Anthony Escalona, and Tariq Kamal. 2016. Skyfire: Model-Based Testing with Cucumber. In *2016 IEEE International Conference on Software Testing, Verification and Validation, ICST 2016, Chicago, IL, USA, April 11-15, 2016*. IEEE Computer Society, 393–400. <https://doi.org/10.1109/ICST.2016.41>
- [13] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn EM van der Werf, and Sjaak Brinkkemper. 2016. Improving agile requirements: the quality user story framework and tool. *Req. Eng.* 21, 3 (2016), 383–403.
- [14] Gabriel Oliveira and Sabrina Marczak. 2017. On the Empirical Evaluation of BDD Scenarios Quality: Preliminary Findings of an Empirical Study. In *IEEE 25th International Requirements Engineering Conference Workshops, RE 2017 Workshops, Lisbon, Portugal, September 4-8, 2017*. IEEE Computer Society, 299–302. <https://doi.org/10.1109/REW.2017.62>
- [15] Gabriel Oliveira and Sabrina Marczak. 2018. On the Understanding of BDD Scenarios' Quality: Preliminary Practitioners' Opinions, See [10], 290–296. https://doi.org/10.1007/978-3-319-77243-1_18
- [16] OMG. 2015. *Unified Modeling Language. Version 2.5*. Technical Report.
- [17] Klaus Pohl. 2016. *Requirements engineering fundamentals: a study guide for the certified professional for req. eng. exam-foundation level-IREB compliant*. Rocky Nook, Inc.
- [18] SEVOCAB. 2015. *Software and Systems Engineering Vocabulary*. (2015).
- [19] Thiago Rocha Silva, Jean-Luc Hak, and Marco Winckler. 2016. Testing prototypes and final user interfaces through an ontological perspective for behavior-driven development. In *Human-Centered and Error-Resilient Systems Development*. Springer, 86–107.
- [20] Monique Snoeck and Yves Wautelet. 2022. Agile MERODE: a model-driven software engineering method for user-centric and value-based development. *Software and Systems Modeling* (2022), 1–26.
- [21] Konstantinos Tsilionis, Yves Wautelet, Charline Faut, and Samedi Heng. 2021. Unifying Behavior Driven Development Templates. In *29th IEEE International Requirements Engineering Conference, RE 2021, Notre Dame, IN, USA, September 20-24, 2021*. IEEE, 454–455. <https://doi.org/10.1109/RE51729.2021.00067>
- [22] Konstantinos Tsilionis, Yves Wautelet, and Samedi Heng. 2022. Building a Unified Ontology for Behavior Driven Development Scenarios. In *Product-Focused Software Process Improvement - 23rd International Conference, PROFES 2022, Jyväskylä, Finland, November 21-23, 2022, Proceedings (Lecture Notes in Computer Science)*, Davide Taibi, Marco Kuhrmann, Tommi Mikkonen, Jil Klünder, and Pekka Abrahamsson (Eds.), Vol. 13709. Springer, 518–524.
- [23] Yves Wautelet, Samedi Heng, Diana Hintea, Manuel Kolp, and Stephan Poelmans. 2016. Bridging User Story Sets with the Use Case Model. In *Advances in Conceptual Modeling - ER 2016 Workshops, AHA, MoBiD, MORE-BI, MReBA, QMMQ, SCME, and WM2SP, Gifu, Japan, November 14-17, 2016, Proceedings (Lecture Notes in Computer Science)*, Sebastian Link and Juan Trujillo (Eds.), Vol. 9975. 127–138.
- [24] Yves Wautelet, Samedi Heng, Soreangsey Kiv, and Manuel Kolp. 2017. User-story driven development of multi-agent systems: A process fragment for agile methods. *COMLAN* 50 (2017), 159–176.
- [25] Yves Wautelet, Samedi Heng, Manuel Kolp, and Isabelle Mirbel. 2014. Unifying and Extending User Story Models. In *Advanced Information Systems Engineering - 26th International Conference, CAiSE 2014, Thessaloniki, Greece, June 16-20, 2014, Proceedings (Lecture Notes in Computer Science)*, Matthias Jarke, John Mylopoulos, Christoph Quix, Colette Rolland, Yannis Manolopoulos, Haralambos Mouratidis, and Jennifer Horkoff (Eds.), Vol. 8484. Springer, 211–225. https://doi.org/10.1007/978-3-319-07881-6_15
- [26] Yves Wautelet, Samedi Heng, Manuel Kolp, Isabelle Mirbel, and Stephan Poelmans. 2016. Building a rationale diagram for evaluating user story sets. In *Tenth IEEE International Conference on Research Challenges in Information Science, RCIS 2016, Grenoble, France, June 1-3, 2016*. IEEE, 1–12. <https://ieeexplore.ieee.org/xpl/conhome/7541913/proceeding>
- [27] Matt Wynne, Aslak Hellesoy, and Steve Tooke. 2017. *The cucumber book: behaviour-driven development for testers and developers*. Pragmatic Bookshelf.